Interactive User Environment Application on Kubernetes Platform

Chalvatzis Kyriakos

TUC

July 9, 2025

Outline

Background

- Data analytics has become essential across all fields — but entry-level users often struggle with the required infrastructure.
- Motivation: Lower the barrier for non-experts to explore and process data using standard tools. User-friendly analytical computing!
- While systems like Apache Spark, Airflow, and Hadoop are powerful, they are built for experienced engineers.
- No current open system provides isolated, user-scoped, web-accessible environments on Kubernetes — tailored for batch analytics.

Goals

- Although not a research thesis, this work aims to prototype a full-stack system that provides:
  - Secure, multi-user access
  - Data upload and sharing capabilities
  - Batch job execution in a Kubernetes-native way
  - Integrated application-like tools such as DuckDB, Python Pandas, Octave and more can be applied on datasets
- Goal: Deliver an introductory platform for data analytics without overwhelming users.
- Goal: Achieve a Cloud-native and modular design.

System Architecture

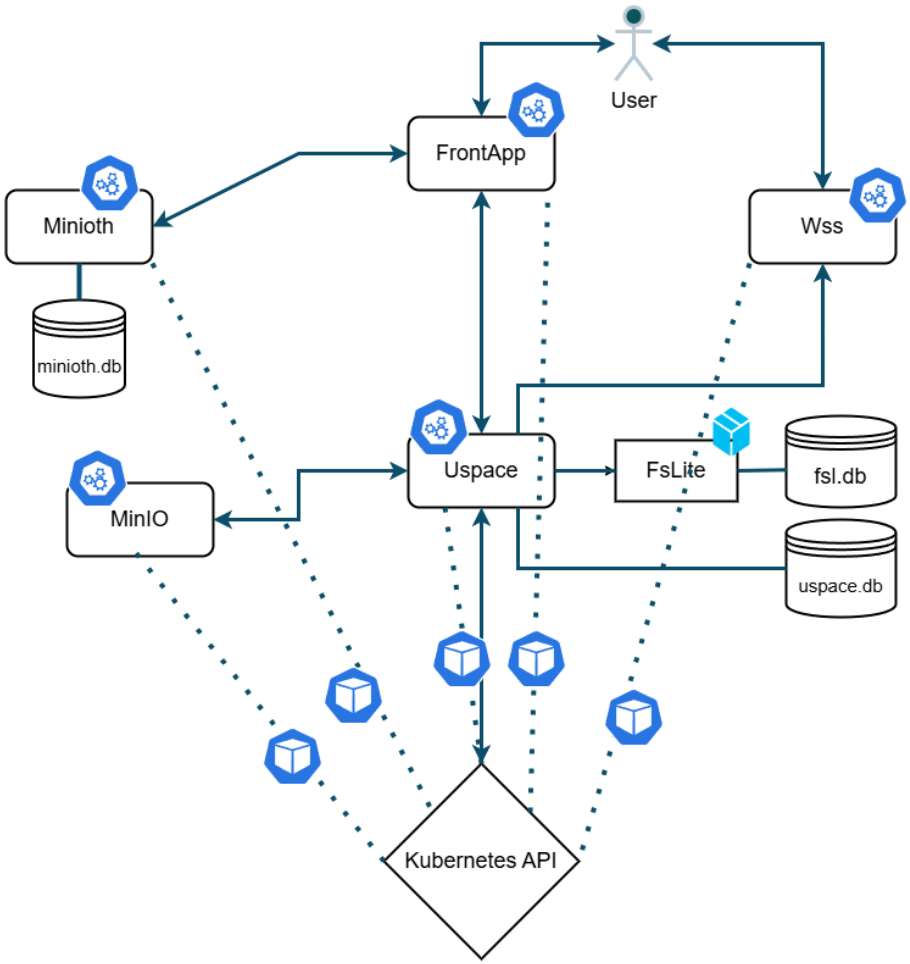
- Microservice-based architecture deployed on Kubernetes
  - Why? "Seperation of Concerts", scalable, modular and adaptable
- Core components:
  - **Uspace API**: User-facing logic for datasets, jobs, resources
  - **FsLite**: Virtual filesystem abstraction with permissions
  - **MinIO**: Object storage backend
  - **Minioth**: Authentication and group-based access control
  - **FrontApp**: Web frontend (HTML templates, HTMX, WebSocket integration)
- Technologies: Go, Gin, Kubernetes, Docker

Goals 2

What we want to achieve in this system:

- Identity - (Authentication)
- Secure Access - (Authorization)
- Physical Storage - (Data Storage)
- Job execution mechanism - (Job Flow)

System Diagram



Uspace Storage Control

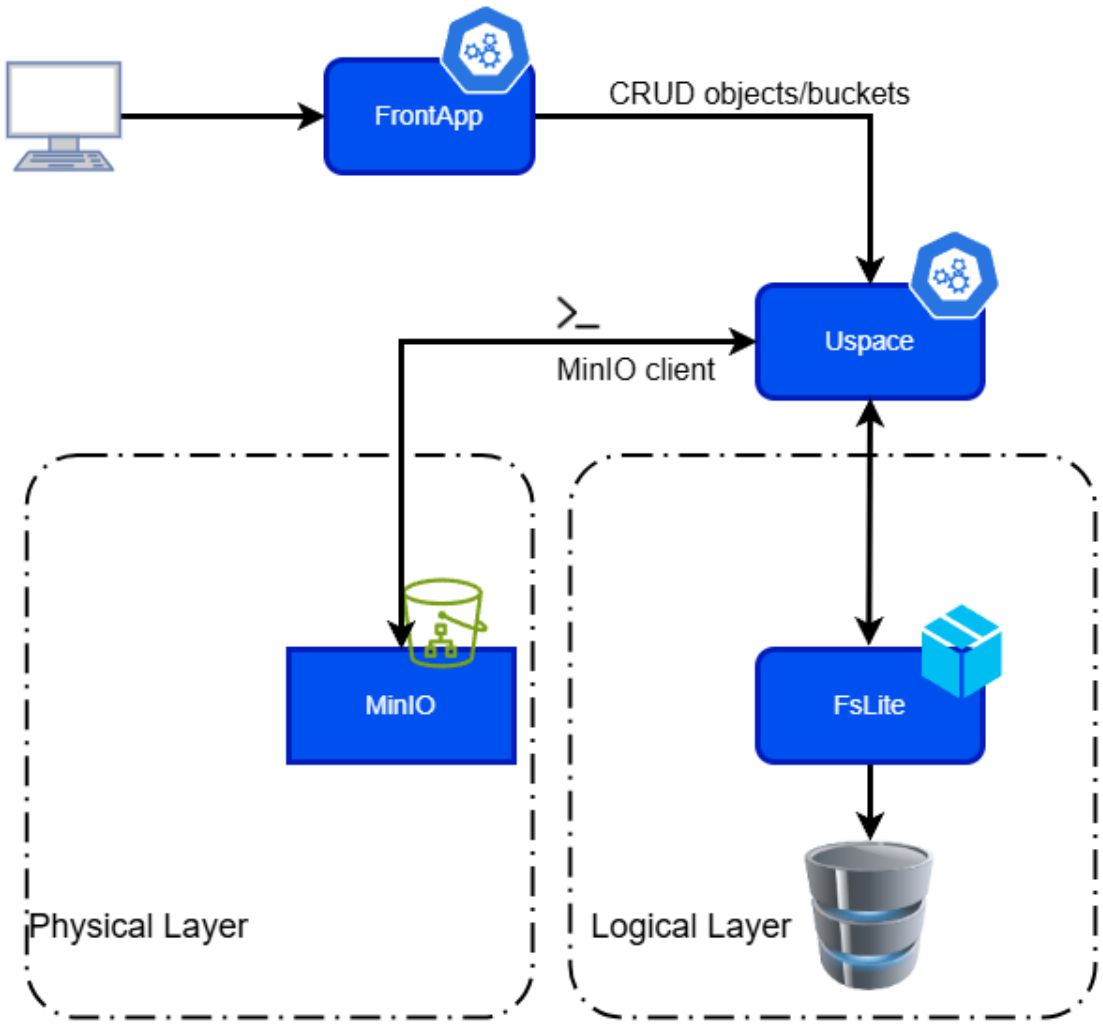


Figure: UspaceStorage

Uspace Job Lifecycle

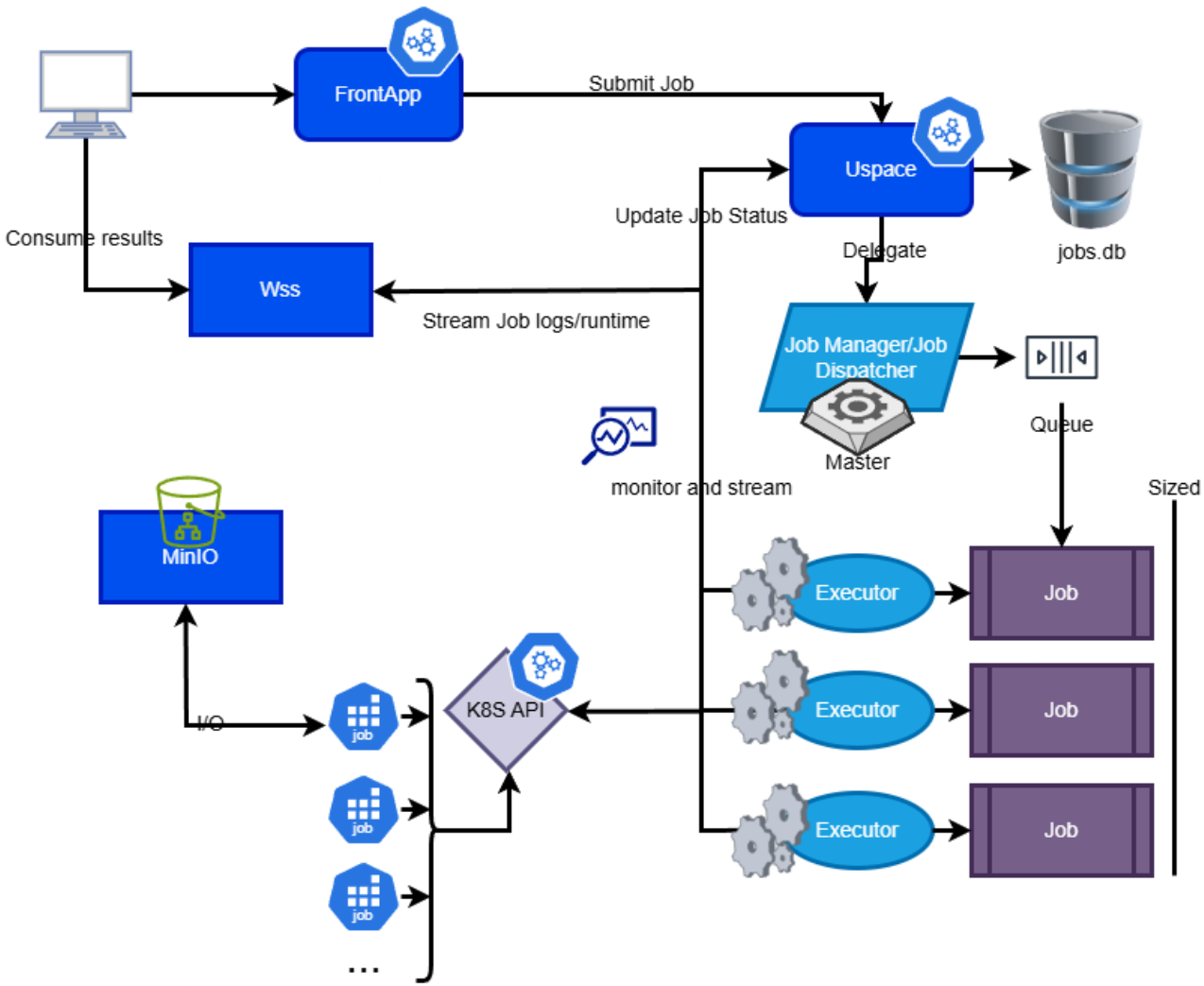
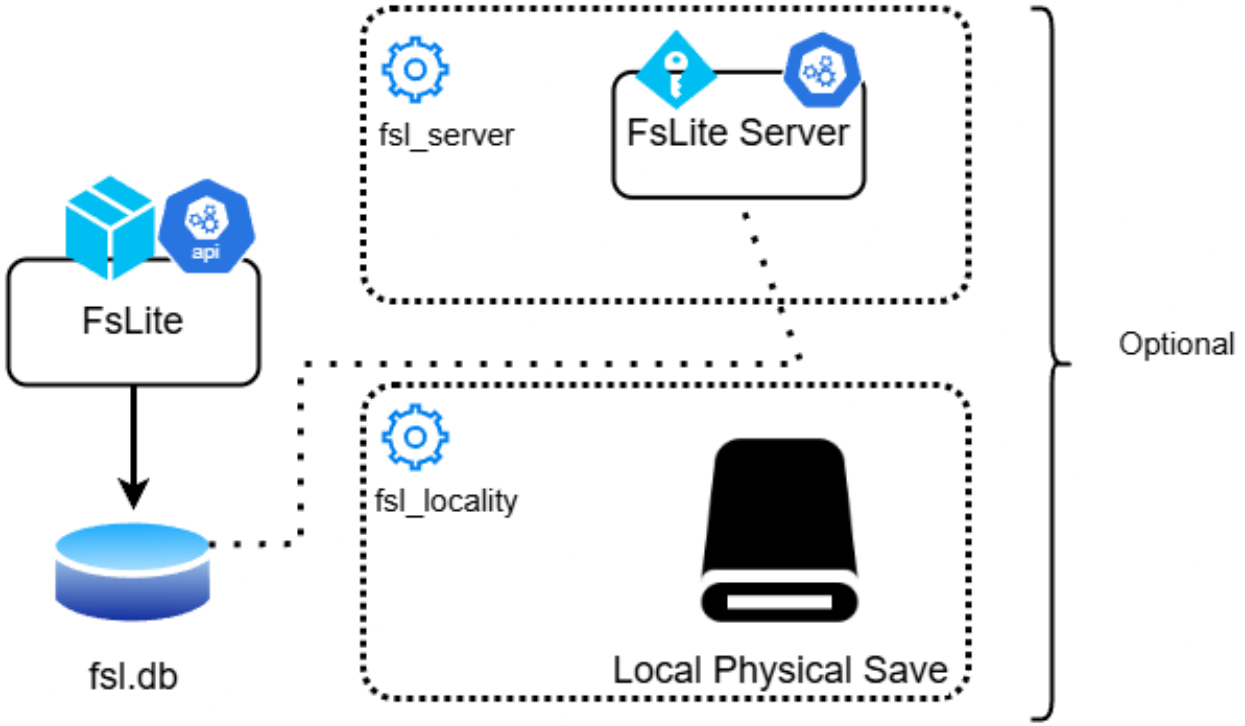
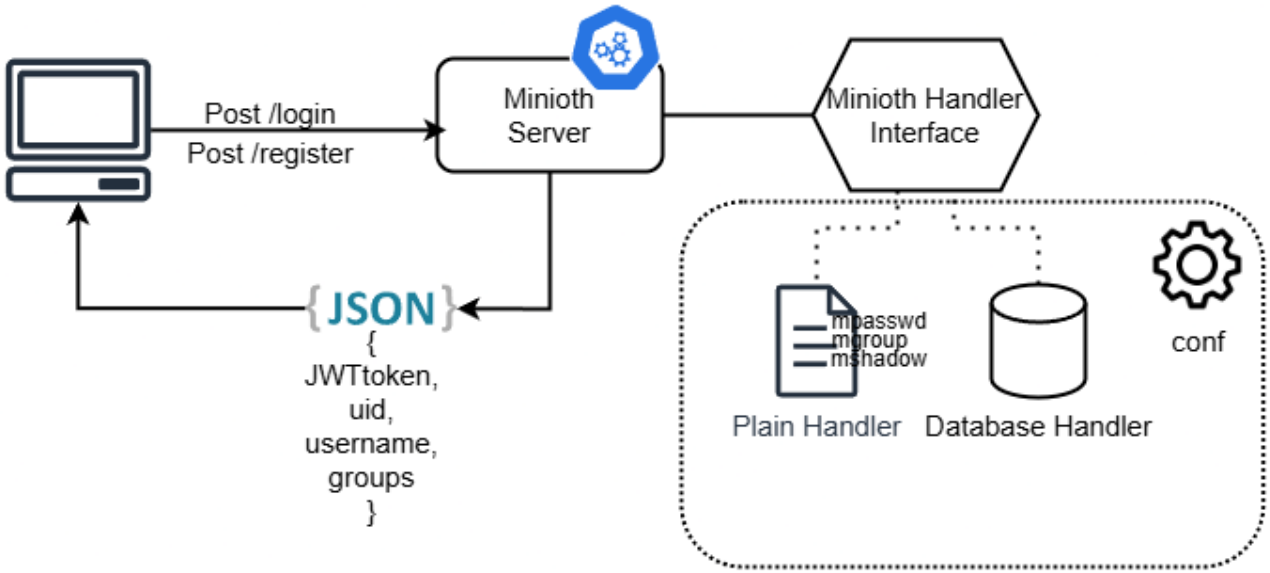


Figure: Uspace Jobs

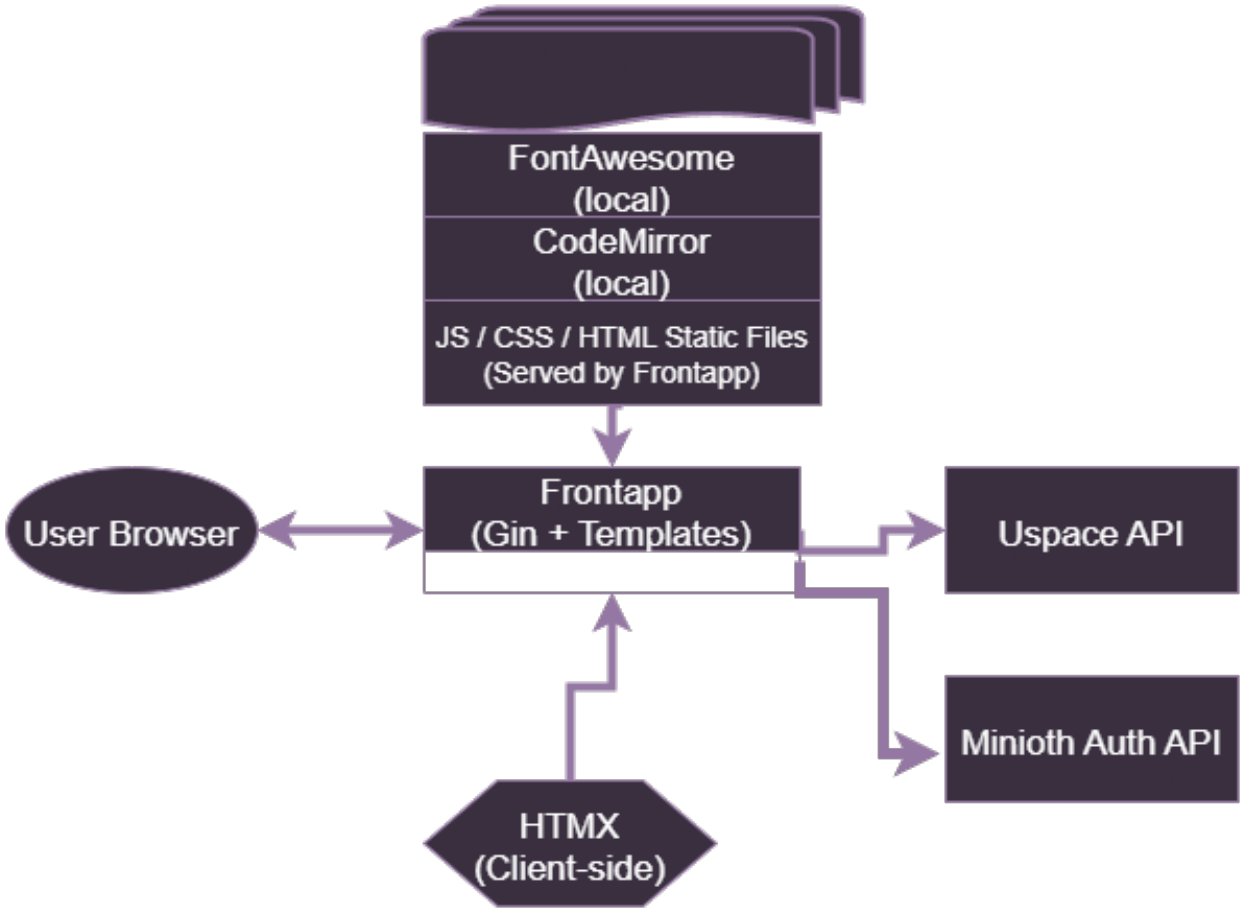
FsLite Structure



Minioth Structure



Frontapp Structure



Uspace API

Method	Path	Description
GET/POST/PUT/DELETE/PATCH	`/api/v1/admin/volumes`	Full volume management API.
DELETE/PUT	`/api/v1/admin/job`	Administrative job control (e.g., delete jobs).
GET/POST/PATCH/DELETE	`/api/v1/admin/user/volume`	Manage user-volume mappings.
GET/POST/PUT/DELETE	`/api/v1/admin/app`	Manage available applications/tools.
GET	`/api/v1/admin/system-conf`	View current system configuration.
GET	`/api/v1/admin/system-metrics`	System metrics via Kubernetes API.

Table: Uspace Admin API Endpoints

FsLite API

Method	Path	Description
POST	`/login`	Authenticate and receive a token.
POST	`/admin/register`	Authenticate and receive JWT token.
POST	`/admin/volume/new`	Change password for the authenticated user.
DELETE	`/admin/volume/delete`	Delete a specified volume
GET	`/admin/volume/get`	Retrieve information about a logical volume
GET	`/admin/resource/get`	Retrieve metadata of multiple resources
GET	`/admin/resource/stat`	Retrieve more detailed metadata of a specific resource.
DELETE	`/admin/resource/delete`	Delete a specified resource.
POST	`/admin/resource/upload`	Upload resource content to volume.
GET	`/admin/resource/download`	Download a resource from storage.
GET/PATCH/DELETE	`/admin/user/voumes`	CRUD user volumes.

Table: FsLite public API endpoints

Minioth API

Method	Path	Description	Auth
POST	`/v1/register`	Register a new user.	No
POST	`/v1/login`	Authenticate and receive JWT token.	No
POST	`/v1/passwd`	Change password for the authenticated user.	Yes (user/admin)
GET	`/v1/user/me`	Get information about the token's user.	Token
GET	`/v1/user/token`	View current token details.	No
POST	`/v1/user/refresh-token`	Request a new access token using a refresh token.	No
GET	`/v1/swagger/*any`	Swagger UI for live API documentation.	No

Table: Minioth public/user API endpoints

Demonstration & Key Features

- Fully functioning web interface for:
  - Uploading datasets
  - Launching batch jobs with SQL, Python, or predefined tools
  - Viewing logs in real time (WebSocket streaming)
- Example: Executing DuckDB query on uploaded CSV $\rightarrow$ result saved and downloadable
- Job isolation: Every job runs in its own Kubernetes Pod

Summary and Future Work

- **Accomplishments**:
  - Designed and implemented a user-centric data platform on Kubernetes
  - Combined identity, authorization, storage, and batch compute layers
  - Delivered a fully working demo with real-world tools and users
- **Limitations**:
  - Not expected to perform fast on really big data
- **Future Work**:
  - Extend toolset: Jupyter, R, Julia, Tool For Tools
  - Add CI/CD for job templates and data pipelines
  - CLI tool, a road to the Shell
  - Distributed orchestration...

Questions?

Thank you!

kchalvatzis@tuc.gr